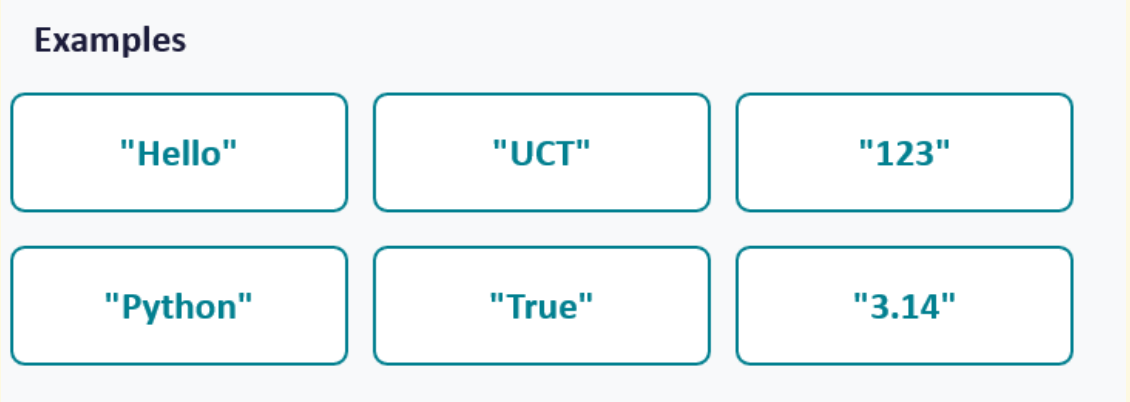
Luvuyo Matomela & Hlomla Kuhlane

"String Conundrum"

1 String Introduction

Strings are essential for :

- **Displaying information to users**
- **Processing input**
- **Creating error messages**
- **Building text-based applications**



2 What's the deal

1 in 3

students fail intro programming globally

50+

years failure rate has not changed

37M

compilations analysed — string errors hardest to fix

At UCT

Student marks consistently drop when string concepts are introduced

Problem Statement

Students repeatedly encounter string concepts without developing correct understanding.

String errors are frequent, persistent, and resistant to traditional instruction, students cannot transfer what they were taught.

Non-intuitive

String immutability contradicts how most beginners think about variables. Modifying a string creates a new object students expect in-place change.

Index Confusion

Zero-based indexing conflicts with everyday counting. Students consistently start counting from 1 and produce off-by-one errors.

Wrong Comparison

String comparison behaves differently from numeric comparison in ways students do not anticipate, causing silent logical errors.



6 Example

```
In [1]:  1 s = 'abcde'
         2 s[0]

Out[1]: 'a'

In [2]:  1 s[0] = '!'     # strings are immutable -- they can never, ever change

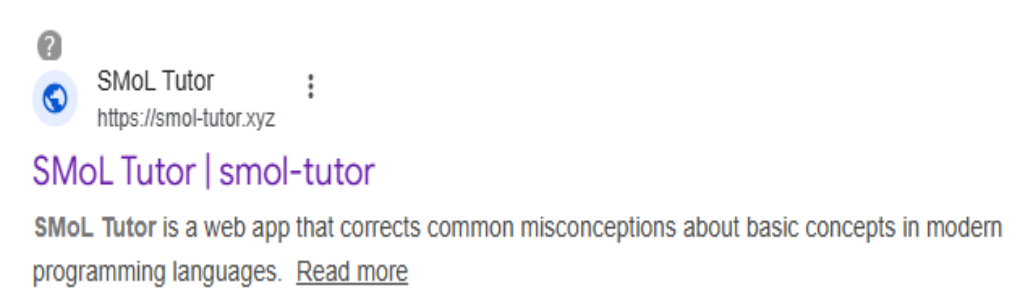
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/var/folders/rr/0mnyyv811fs5vyp22gf4fxk00000gn/T/ipykernel_10770/3916650957.py in <module>
----> 1 s[0] = '!'

TypeError: 'str' object does not support item assignment
```

- **assumes the student understands *"object"***
- **doesn't explain immutability**
- **doesn't tell them what they did wrong in simple terms**

7 Issues with Existing Tools





The Gap

- **No tool targets string misconceptions specifically**
- **Behaviour is made visible but never explained**
- **No misconception-targeted feedback for strings**
- **Students must infer understanding on their own**

Project Aims

8 Project Aims

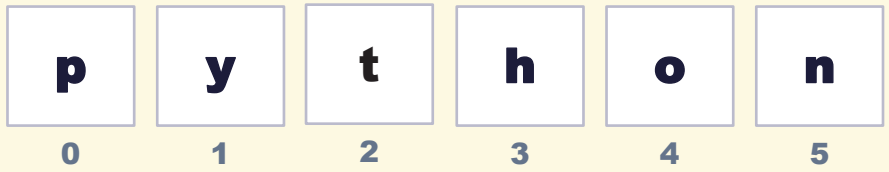


Visualize Strings	Display strings as indexed character sequences with dynamic highlighting during operations
Live Code Pipeline	Accept Python string code, execute it, and transform the result into the visual representation
Structured Progression	Present string concepts from fundamental to complex using multiple-choice and fill-in-the-blank exercises
Targeted Feedback	Identify the specific misconception behind every wrong answer and explain it in plain language
Evaluate Learning	Measure learning gain through pre and post testing with UCT first-year Python students

Feature Concepts

9 Basic Example

word

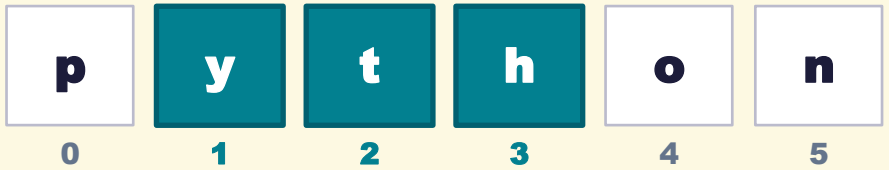


word[2] — single character highlight



→ Index 2 highlights

word[1:4] — slice highlight



→ Boxes 1, 2, 3 highlight | Box 4 remains unhighlighted (exclusive end bound)

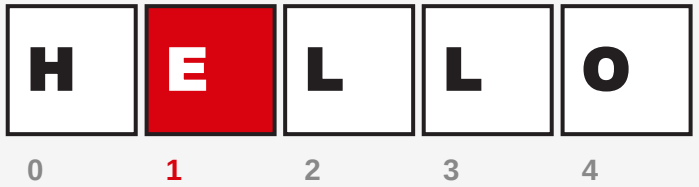
10 Live Code Pipeline

INPUT · Python

```
s = "HELLO"
print(s[1])
```



OUTPUT · Visualiser



Result: "E"

1 Receive

Code POSTed to Flask

2 Parse

Python AST validates code

3 Sandbox

Subprocess runs with CPU/mem limits

4 Visualise

Returns JSON string state

11 Learning View

MODULE 2 · IMMUTABILITY

What is printed by the code below?

```
s = "cat"
s[0] = "b"
print(s)
```

A "bat" ✗

B "cat"

C TypeError ✓

D "b"

MISCONCEPTION-TARGETED FEEDBACK

Strings cannot be modified in place

Your answer suggests strings work like lists — that you can change one character. In Python, strings are immutable: writing s[0] = "b" raises a TypeError. To change a string, build a new one:

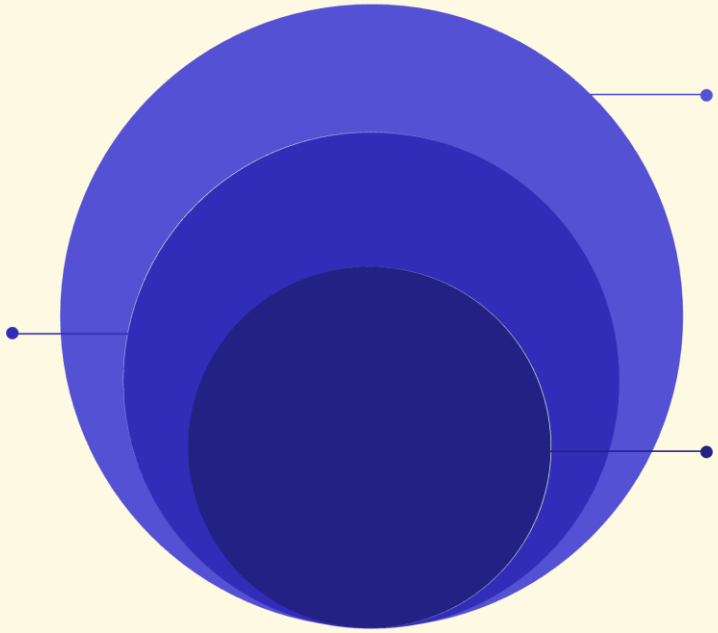
```
s = "b" + s[1:]
print(s)  # "bat"
```

Try again →

Learning programming requires the construction of appropriate mental models of program execution

Amal Khaled Altadmri & Neil C. C. Brown (2015)

12 System Architecture



View

React frontend rendering UI and string visualizations

Controller

Flask backend handling logic and code execution

Model

Firebase DB storing user progress and results

Hosted on UCT CS Server (Nightmare)

13 Ethics & Legal

ETHICS APPROVED · UCT · FACULTY OF SCIENCE

Compliance Checklist

Ethical, Professional & Legal Issues · submission deadline 11 May 2026

✓ **REC clearance**	Faculty of Science Research Ethics Committee approval obtained before any participant recruitment or testing begins.
✓ **Informed consent**	Voluntary participation; signed consent form; participants may withdraw at any stage without consequence.
✓ **Anonymised data**	No personally identifying information stored. Progress and answer data anonymised, used solely for evaluation.
✓ **Open-source IP**	Intellectual property owned by UCT per university policy. Source code released open-source via Research Contracts.

14 Evaluation

Functional Testing

01

Unit tests on every REST endpoint and sandbox boundary, run on every commit via GitHub Actions

Usability Testing

02

15–20 first-years in think-aloud sessions. SUS score benchmarked against published thresholds.

Comparative Learning

03

Pre/post-test with experimental and control groups. T-test with reported Cohen's d effect size.

Performance & Stability

04

Locust load tests verify 20 concurrent users at sub-2-second response time before recruitment.

15 Expected Outcomes

01

TANGIBLE

Deployed web app visualiser, code pipeline, exercises, feedback engine

02

ACADEMIC

Improved student performance in string-related coursework at UCT

03

COGNITIVE

Students develop correct mental models of string behaviour

04

COMMUNITY

Tool released openly for adoption by other institutions

16 Key Success Factors

How we'll know we've succeeded, measurable criteria, set up front

MODULES

100%

Full Deployment

All string concept modules functional and accessible through any browser, no install required

SUS SCORE

>68

Usability Threshold

Students rate the tool above the established SUS benchmark in first-year usability testing

SIGNIFICANCE

p<0.05

Learning Gain

Statistically significant pre/post-test improvement reported with Cohen's d effect size

ACCURACY

100%

Targeted Feedback

Feedback engine identifies the correct misconception for every wrong answer across modules

Top 4 Project Risks

Feature creep (High)
Stick to five core aims

1

Academic workload delays (High)
Weekly project time; track Gantt; descope if needed

2

Ethics clearance delay (Medium)
prep instruments in advance

3

Backend execution security (Medium)
Subprocess sandbox with CPU/mem limits; block unsafe imports

4

17 Timeline



Initial setup of the entire system and UI mockups

Development cycle 1. Main Features completed.

Development cycle 2 including. Including all 3-4 modules + User Testing

Development cycle 3. Improving on application + including User Testing

Final code Submission 10 September 2026

18 team work distribution

Luvuyo Matomela
Backend



- Develops the backend system and data pipeline
- Implements the sandboxed Python execution engine
- Builds misconception detection
- Generates targeted feedback for student errors
- Manages Firestore database operations and scoring systems
- Designs REST APIs for communications



Hlomla Kuhlane
Frontend

- Develops all user-facing components using React
- Designs UI and user flows
- Implements string visualisation and interactions
- Manages user input, navigation, and feedback display
- Builds dashboard, scoring, and leaderboard features
- Integrates Firebase authentication (login & signup)

19 Summary

- String Conundrum addresses a key challenge in programming education: students struggle to understand string concepts at a conceptual level

- Existing tools provide partial support but fail to target string-specific misconceptions

Our solution introduces a web-based interactive system that combines:

- Visualisation
- Code execution
- Structured learning
- Targeted feedback

The system is designed to help students build correct mental models, not just produce correct code

Through evaluation, we aim to demonstrate improved understanding, performance, and confidence

Feature Creep: Given enough time a transpiler will be added to compile from python to isiPython

Thank you

Mtmluv001@myuct.ac.za

Khlhlo002@myuct.ac.za